

Course Outline: Payload Optimization

Title: Payload Optimization **Skill:** Skill 37: Optimizar la arquitectura para alto tráfico empleando Caching
Learnpack: + Optimización de payload **File:** s37-w13d38-optimizacion-de-payload **Prerequisite:**
Serializers (s36-w13d37-serializers) **Target Audience:** Beginner developers building FastAPI backends
Total Lessons: 7 **Format:** Mixed (14% hands-on = 1 lesson)

Course Description

Returning too much data is one of the most common and easiest-to-fix performance problems in API development. In this course, students go beyond the brief introduction in the Serializers learnpack and learn how to design APIs that return exactly what each screen needs — nothing more. They'll implement field-selective endpoints using FastAPI response models, and understand when GraphQL offers a structural alternative to the same problem.

Course Philosophy

This course emphasizes:

- Precision over completeness: an API response is a contract, not a dump
 - Performance as a design decision, not an afterthought
 - Understanding trade-offs: REST with response models vs. GraphQL's flexible queries
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Payload Optimization	4
02 - Assessment	1
03 - Outro	1
Total	7

00.0 Welcome

Learning Objectives:

- Understand what this course covers and why payload size matters in API performance
- Preview the practical skill of returning field-selective responses in FastAPI

Content Outline:

- The hidden cost of sending too much data
- What students will be able to do by the end of this course
- Course section overview

Transitions: Leads into Section 01, where we explore what payload optimization means and why it matters at scale.

Key Principles:

- An API that returns what every possible consumer might ever need is not a good API — it's a lazy one
- Performance optimization starts with what you choose NOT to send

Examples:

- A mobile app screen that shows only a user's name and avatar — but the endpoint returns the user's full profile including address, payment history, and preferences
 - The unnecessary data adds latency, increases bandwidth, and slows down the client
-

01.0 Payload Optimization

Learning Objectives:

- Explain what payload optimization means in the context of API design
- Identify the overfetching problem and recognize it in real API responses
- Preview the three sub-lessons that cover how to solve it

Content Outline:

- What is a payload in API communication?
- Overfetching: returning more than the consumer needs
- Why it matters: latency, bandwidth, mobile performance
- Two approaches to the problem: REST with selective responses, and GraphQL
- What the upcoming lessons will cover

Transitions: This overview frames the problem. Lesson 01.1 dives into the solution — field selection via FastAPI response models. Lesson 01.2 then compares that approach with GraphQL. Lesson 01.3 gives hands-on practice.

Key Principles:

- "Only return exactly what the current screen needs" — the core rule of payload optimization
- Overfetching is a design smell, not a runtime bug: it's caught in code review, not crash logs
- The cost is paid by the slowest client, not the fastest server

Examples:

- A `/users/{id}` endpoint returning 20 fields when the front-end only uses 3
- A dashboard that makes 5 API calls, each returning full objects, when it could make 1 targeted call

01.1 Returning What the Screen Needs

Learning Objectives:

- Use Pydantic response models to control which fields a FastAPI endpoint returns
- Distinguish between a database model, a DTO, and a response model
- Design an endpoint that returns a field subset tailored to a specific consumer

Content Outline:

- Review: why the DB model should never be serialized directly to the frontend (covered in Serializers)
- Response models as output DTOs: shape the data for the consumer, not the database
- Using `response_model=` in FastAPI to declare what gets returned
- Field projection: defining a Pydantic schema with only the needed fields
- Multiple response models for different consumers (e.g., `UserCard` for a list view vs. `UserDetail` for a detail page)

Transitions: This lesson establishes the server-side solution. Lesson 01.2 presents GraphQL as a client-side alternative to the same problem.

Key Principles:

- Response models are consumer contracts: they describe what ONE screen needs, not what the database has
- If two screens need different data from the same resource, define two response models — not one that satisfies both
- The `response_model=` declaration in FastAPI is also documentation: it tells the API consumer exactly what to expect

Examples:

```
# Full DB model – never return this directly
class UserDB(Base):
    id, email, password_hash, address, bio, created_at, updated_at, ...

# Response model for a profile card (only what the screen needs)
class UserCard(BaseModel):
    id: int
    name: str
    avatar_url: str

# Endpoint using the response model – FastAPI strips everything else
@app.get("/users/{user_id}", response_model=UserCard)
def get_user_card(user_id: int):
    return db.query(User).filter(User.id == user_id).first()
```

01.2 GraphQL vs Optimized REST

Learning Objectives:

- Explain how GraphQL lets clients specify exactly which fields they want
- Compare GraphQL's client-driven field selection with REST's server-defined response models
- Identify when each approach is the right tool for the job

Content Outline:

- The core GraphQL insight: let the client declare the shape of the response
- A minimal GraphQL query example requesting specific fields
- How optimized REST (with response models) solves the same problem server-side
- Trade-offs: GraphQL's flexibility vs. REST's simplicity and HTTP caching
- When to choose each: stable consumer needs → REST; diverse or frequently-changing consumers → GraphQL

Transitions: With the concept understood in both forms, lesson 01.3 gives hands-on practice building a selective REST endpoint.

Key Principles:

- Both GraphQL and optimized REST solve overfetching — they make different trade-offs
- GraphQL shifts field selection to the client; REST with response models keeps it on the server
- Neither is universally better: the right choice depends on who your consumers are and how many there are

Examples:

```
# GraphQL query – client asks for exactly these fields
query {
  user(id: 1) {
    name
    avatarUrl
  }
}
```

```
# REST with response model – server defines the contract
GET /users/1
→ { "id": 1, "name": "Ana", "avatar_url": "..."}

```

01.3 Selective API Response

Challenge Prompt: Build a FastAPI endpoint for a user profile card that returns only the fields a mobile screen needs, using a Pydantic response model to restrict the output.

Solution Code:

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class UserInDB(BaseModel):
    id: int
    name: str
    email: str
    password_hash: str
    address: str
    bio: Optional[str] = None
    avatar_url: str

class UserCard(BaseModel):
    id: int
    name: str
    avatar_url: str

fake_db = {
    1: UserInDB(
        id=1, name="Ana", email="ana@example.com",
        password_hash="hashed", address="123 Main St",
        bio="Developer", avatar_url="https://example.com/ana.jpg"
    )
}

@app.get("/users/{user_id}", response_model=UserCard)
def get_user_card(user_id: int):
    return fake_db[user_id]
```

Challenge Code:

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class UserInDB(BaseModel):
    id: int
    name: str
    email: str
    password_hash: str
    address: str
    bio: Optional[str] = None
```

```

    avatar_url: str

# TODO: Define a UserCard response model with only: id, name, avatar_url
class UserCard(BaseModel):
    pass

fake_db = {
    1: UserInDB(
        id=1, name="Ana", email="ana@example.com",
        password_hash="hashed", address="123 Main St",
        bio="Developer", avatar_url="https://example.com/ana.jpg"
    )
}

# TODO: Create a GET /users/{user_id} endpoint that uses
response_model=UserCard

```

02.0 Payload Optimization Assessment 🧐

Learning Objectives:

- Demonstrate understanding of overfetching and its impact on API performance
- Apply knowledge of FastAPI response models to identify correct implementations
- Evaluate REST vs. GraphQL trade-offs in realistic scenarios

Question format: Scenario-based multiple choice

Questions:

1. A FastAPI endpoint for a product listing page currently returns all 15 fields from the **Product** database model. The page only displays **name**, **price**, and **thumbnail_url**. What is the best fix?
 - a) Add a query parameter **?fields=name,price,thumbnail_url** and filter manually in the handler
 - b) ☒ Create a **ProductCard** Pydantic model with only the three needed fields and use **response_model=ProductCard**
 - c) Remove the unused fields from the database model
 - d) Return the full model but document the unused fields as "deprecated"
2. A developer says: "I'll use one response model for all endpoints — that way I only have to maintain one schema." What is the primary problem with this approach?
 - a) Pydantic doesn't support reusing models across endpoints
 - b) ☒ It forces every consumer to receive fields they don't need, reintroducing overfetching
 - c) FastAPI requires a unique response model per endpoint
 - d) It makes the code harder to read
3. Your API serves two clients: a mobile app and an admin dashboard. The mobile app needs **id**, **name**, **avatar_url**. The admin dashboard needs **id**, **name**, **email**, **created_at**, **role**. What is the recommended approach?
 - a) Return all fields and let each client ignore what it doesn't need
 - b) ☒ Create two separate response models: **UserCard** and **UserAdmin**, and use the appropriate one per endpoint
 - c) Use GraphQL instead, since REST can't handle this scenario
 - d) Let the client pass a **fields=** query parameter

4. Which scenario is the BEST fit for GraphQL over optimized REST with response models? a) A simple CRUD API with three well-known clients that rarely change their data needs b) ☒ A public API consumed by dozens of different third-party integrations, each requiring a different subset of fields c) An internal API used by only a mobile app d) Any API that already uses Pydantic
5. A `response_model=UserCard` declaration in FastAPI does which of the following? a) Filters the database query to only fetch the specified columns b) Validates that the input body matches UserCard c) ☒ Serializes the handler's return value to match only the fields defined in UserCard, stripping any extras d) Raises an error if the handler returns extra fields not in UserCard

Evaluation criteria:

- 5/5: Strong grasp of payload optimization, response models, and REST vs. GraphQL trade-offs
- 4/5: Solid understanding with a minor gap
- 3/5 or below: Review lessons 01.1 and 01.2

Score interpretation:

- 5/5 — Excellent. You can now design lean, consumer-driven APIs.
- 4/5 — Good. One concept needs a second look — review the lesson that covers it.
- 3/5 or below — Return to the course content before moving on.

03.0 Keep Building Lean APIs

Content Outline:

- Course recap: what students accomplished
- Connection to bigger picture
- Next steps and continued learning
- Resources for further exploration
- Encouragement and celebration

Note: This is conclusion only — no theory, exercises, or assessment.